

MethylPipeline Overview

Architecture, workflows, and package responsibilities

OpenAI Codex

2026-05-27

Table of contents

1	Executive Summary	1
2	What The Repository Contains	2
2.1	Core Statistical Path	2
2.2	Downstream Interpretation And Support Packages	2
3	Package Architecture	2
4	Shared Data Model	3
5	Main Workflows	3
5.1	Workflow 1: Model Creation	3
5.1.1	Workflow 1 Diagram	4
5.2	Workflow 2: Backend Model-Selection Monte Carlo	5
5.2.1	Workflow 2 Diagram	5
5.3	Workflow 3: Frozen-Artifact Holdout Evaluation	5
6	Statistical Character	5
7	Typical Outputs	6
8	Rendering Notes	6

1 Executive Summary

MethylPipeline is a monorepo for DNA methylation analysis. Its core supervised path centers on empirical cumulative distribution functions (ECDFs) and histogram-backed centroid summaries rather than forcing the full workflow into a single parametric family. Around that core, the repository adds packages for gene mapping, enrichment, prediction, repeated validation, and alignment quality control.

At a practical level, the repository supports three major modes of work:

1. building a stable production model from repeated train-validation splits, and
2. performing backend model-selection Monte Carlo (`--model-mc`) over frozen production context, and
3. applying frozen artifacts for holdout evaluation (`--post-model-validation` and `--predictor-only`).

The codebase is organized as cooperating packages with `methyutils` as the shared mathematical and utility layer.

2 What The Repository Contains

2.1 Core Statistical Path

The central supervised pipeline is:

1. `methycentroid` builds cohort summaries from per-sample methylation values.
2. `methyldetector` screens loci for differential methylation and exports classifier-ready DMP panels.
3. `methyclassifier` fits or applies classification logic over the detected loci.
4. `methypredictor` evaluates trained models on holdout or blind samples.
5. `methyvalidation` orchestrates repeated validation, stability analysis, production freeze, and final model creation.

This path is the most tightly coupled statistical spine of the repository.

2.2 Downstream Interpretation And Support Packages

- `methymapper` maps DMPs to genes and genomic regions.
- `methylenricher` performs enrichment analysis, graph clustering, and module ranking.
- `methyldiseaseprogression` aggregates staged mapper/enricher outputs into progression summaries and labels.
- `methyalignmentqc` parses alignment/QC outputs from upstream sequencing workflows.
- `methyutils` provides shared data structures, ECDF reconstruction, overlap metrics, tests, logging, configuration utilities, and classifier support code.

3 Package Architecture

The package graph is easier to understand as a layered system: shared utilities at the bottom, the supervised methylation workflow in the center, and biological interpretation or QC around it.

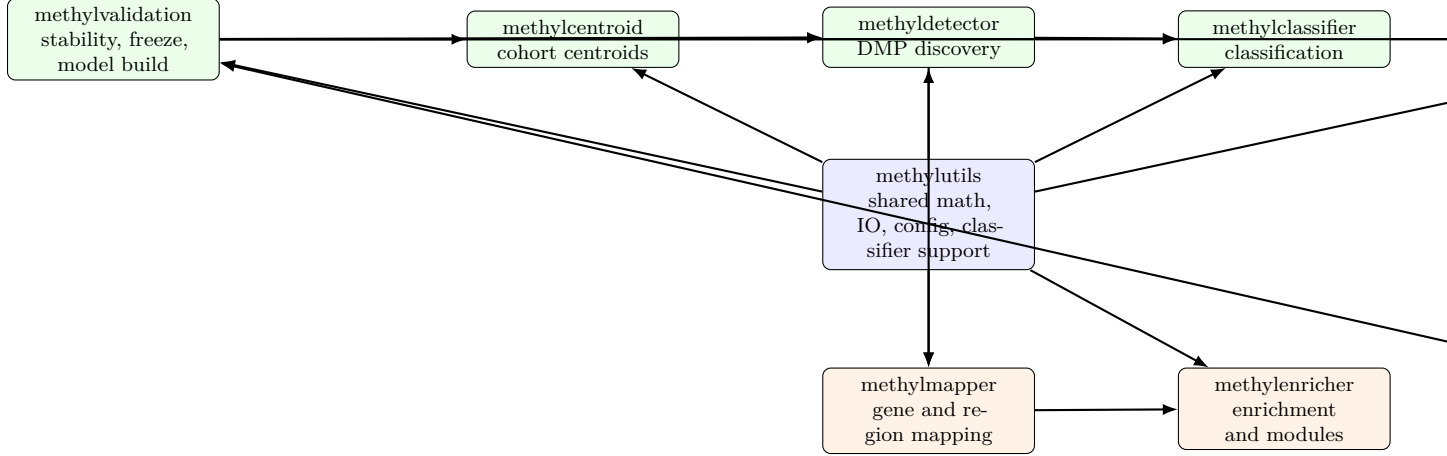


Figure 1: High-level package architecture of MethylPipeline.

4 Shared Data Model

At its lowest common layer, the code works with methylated and unmethylated counts at genomic loci. For sample s and position i ,

$$x_{si} = \frac{m_{si}}{m_{si} + u_{si}},$$

where m_{si} is the methylated count and u_{si} is the unmethylated count.

Centroid-building packages summarize cohorts through per-locus sufficient statistics:

$$N_i = \sum_s 1, \quad S_{x,i} = \sum_s x_{si}, \quad S_{x^2,i} = \sum_s x_{si}^2.$$

These support the implemented cohort mean and variance estimates:

$$\hat{\mu}_i = \frac{S_{x,i}}{N_i}, \quad \hat{\sigma}_i^2 = \max\left(\frac{S_{x^2,i} - S_{x,i}^2/N_i}{\max(N_i - 1, 1)}, \varepsilon\right).$$

In the ECDF-centered path, each locus also stores histogram counts over fixed methylation bins. Those histograms are used to reconstruct empirical distribution views, approximate overlap metrics, and rank-based or distribution-based comparisons at scale.

5 Main Workflows

5.1 Workflow 1: Model Creation

Model creation is the longer production workflow. It is intentionally separated into three stages:

1. `--stability`: repeated train-validation splits that rerun centroid and detector on the training subset only.
2. `--freeze`: rerun the stable panel on all samples and produce downstream biological interpretation.
3. `--model-mc`: run backend model-selection Monte Carlo from frozen production context.
4. `--select-best-model`: rank backends and train final all-data production model.

This split matters statistically because it reduces leakage between feature discovery and final model construction, even though it is still an internal workflow rather than external validation on an independent cohort.

Canonical command sequence:

```

methyl-validation --project configs/my_project.json --stability
methyl-validation --project configs/my_project.json --freeze
methyl-validation --project configs/my_project.json --model-mc --model-mc-all
methyl-validation --project configs/my_project.json --select-best-model --model-mc-all

```

5.1.1 Workflow 1 Diagram

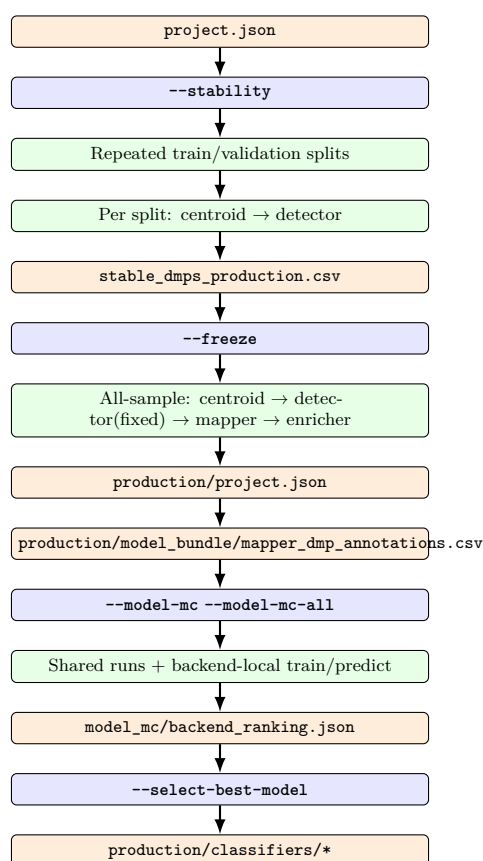


Figure 2: Workflow 1 for stable production model creation with explicit freeze and model-MC artifacts.

5.2 Workflow 2: Backend Model-Selection Monte Carlo

The second workflow assumes a frozen production project already exists and evaluates backend model behavior across repeated holdout splits:

- `--model-mc --model-backend <backend>` evaluates one backend.
- `--model-mc --model-mc-all` builds shared split+centroid+detector runs once, then executes backend-local model stages.
- when a split is compatible with primary MC runs, centroid/detector artifacts are symlinked instead of recomputed.

5.2.1 Workflow 2 Diagram

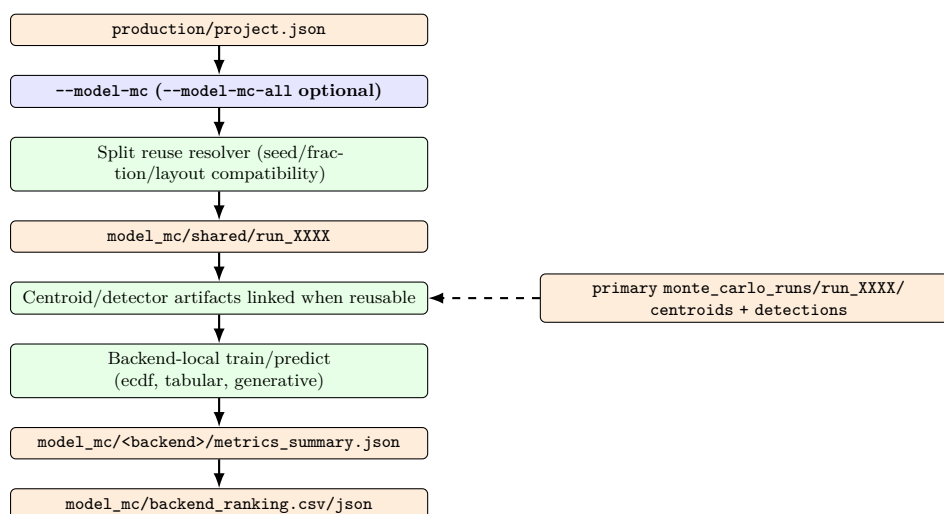


Figure 3: Workflow 2 backend model-selection Monte Carlo with explicit split/artifact reuse path.

5.3 Workflow 3: Frozen-Artifact Holdout Evaluation

This workflow evaluates frozen artifacts without retraining:

- `--post-model-validation` runs Monte Carlo holdout evaluation using frozen production artifacts (backend-aware dispatch).
- `--predictor-only` executes repeated predictor-only runs that reuse frozen paths.

It answers a deployment-style question: how stable are frozen-model metrics across sampled validation subsets when training is fixed.

6 Statistical Character

MethylPipeline mixes several methodological categories, and the documentation should keep them distinct:

- principled methods such as standard hypothesis tests, multiple-testing correction, and established classifiers,
- approximate methods where empirical distributions are reconstructed from summaries or discretized histograms,
- heuristic rules for ranking, thresholding, or aggregation,
- external-service-backed steps in downstream interpretation layers.

That distinction is important because the repository is strongest when it speaks precisely about what is implemented. The core centroid-detector-classifier path is grounded in empirical-distribution summaries, while downstream interpretation layers may depend on graph heuristics, curated databases, or external enrichment services.

7 Typical Outputs

Common outputs across the repository include:

- centroid files summarizing cohort-level methylation behavior,
- detector outputs containing DMP statistics and filtered panels,
- classifier artifacts and predictor metrics,
- stability summaries across Monte Carlo iterations,
- model-MC shared/backend roots with split-reuse diagnostics and backend ranking,
- mapping and enrichment outputs for downstream biological interpretation,
- mapper annotation cache files under `production/model_bundle/` for mapped-family model builds,
- aggregated ECDF OvR artifacts (`ecdf_aggregated_ovr.pkl` + `.meta.json`) for observed-hybrid ECDF mode,
- QC summaries extracted from external alignment reports.

A typical production-oriented run produces a `monte_carlo_runs/production/` tree containing the frozen project, stable panel outputs, downstream interpretation products, and final deployable model artifacts.

8 Rendering Notes

This file is intended to render directly with Quarto on a machine that has LaTeX support available. The PDF format is configured to use `lualatex`, and the TikZ diagrams are injected through `include-in-header`.

Example commands:

```
quarto render docs/MethylPipeline-overview.qmd --to html
quarto render docs/MethylPipeline-overview.qmd --to pdf
```

If your Quarto installation already has TinyTeX, TeX Live, or MiKTeX available, the PDF build should render the TikZ diagrams without extra changes.